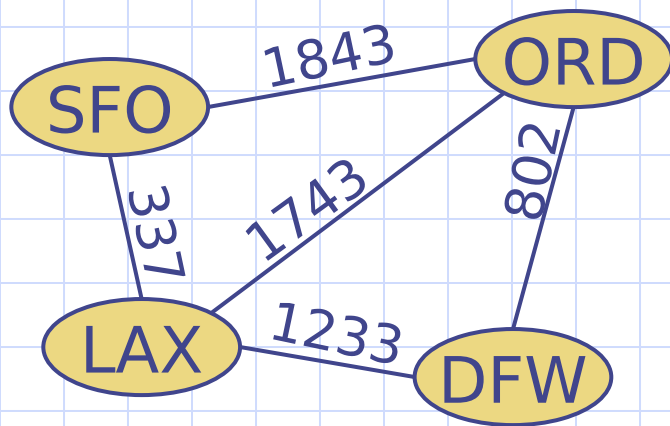
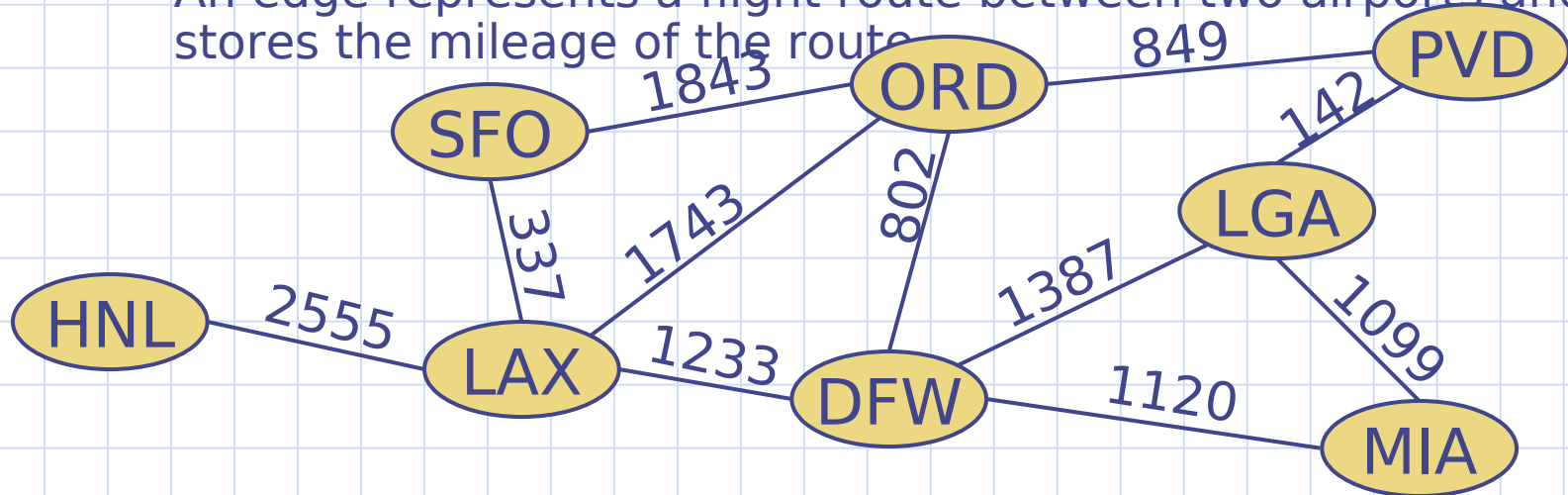


Graphs



Graphs

- A graph is a pair (V, E) , where
 - V is a set of nodes, called **vertices**
 - E is a collection of pairs of vertices, called **edges**
 - Vertices and edges are positions and store elements
- Example:
 - A vertex represents an airport and stores the three-letter airport code
 - An edge represents a flight route between two airports and stores the mileage of the route



Edge Types

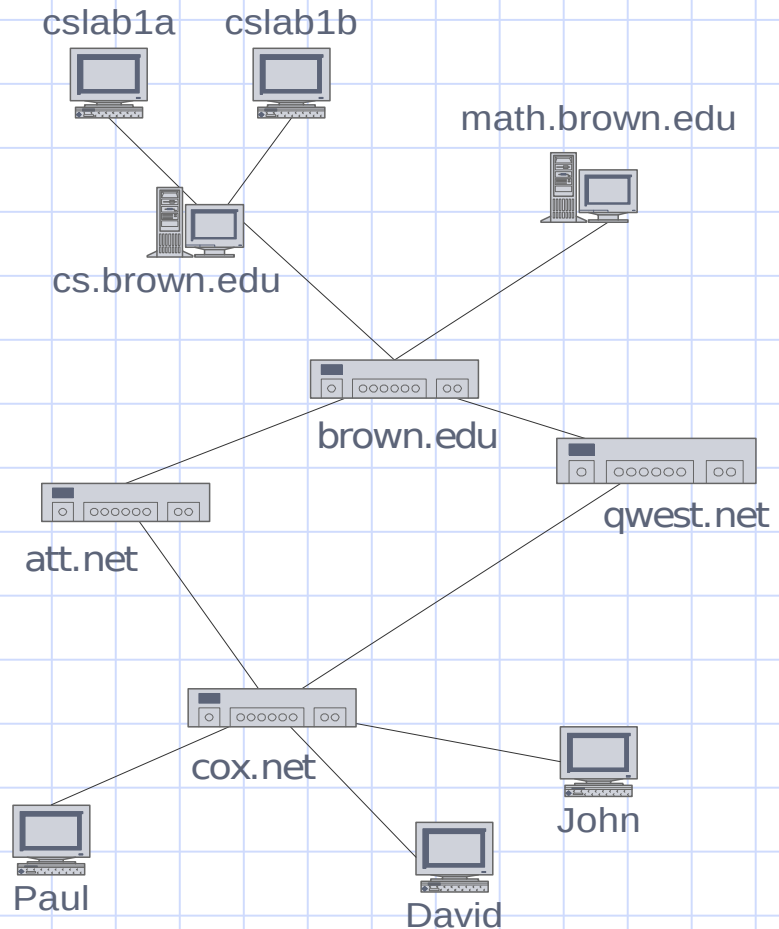
- Directed edge
 - ordered pair of vertices (u,v)
 - first vertex u is the origin
 - second vertex v is the destination
 - e.g., a flight
- Undirected edge
 - unordered pair of vertices (u,v)
 - e.g., a flight route
- Directed graph
 - all the edges are directed
 - e.g., route network
- Undirected graph
 - all the edges are undirected
 - e.g., flight network



Applications

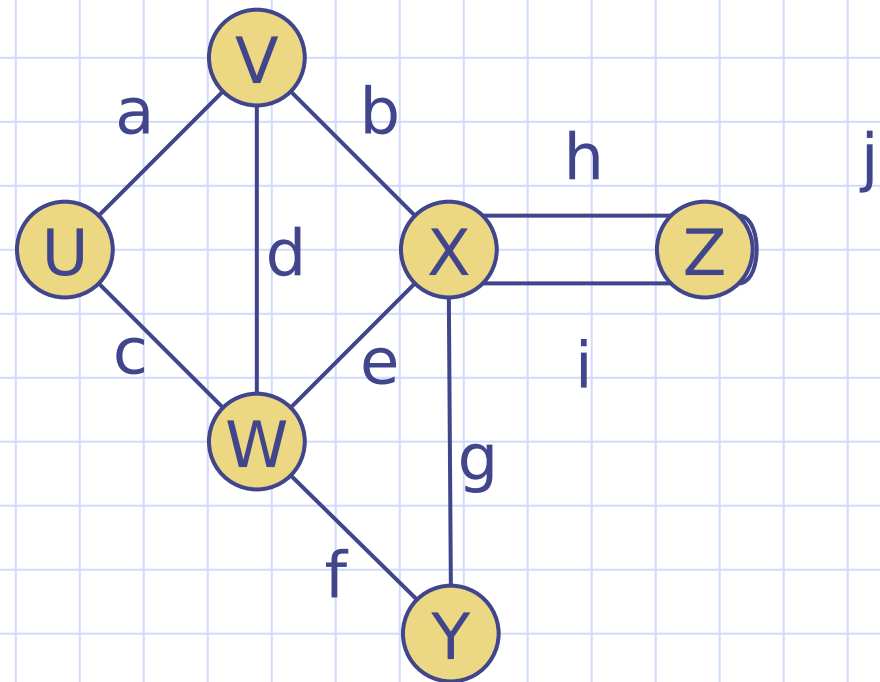
- ❑ Electronic circuits
 - Printed circuit board
 - Integrated circuit
- ❑ Transportation networks
 - Highway network
 - Flight network
- ❑ Computer networks
 - Local area network
 - Internet
 - Web
- ❑ Databases

- Entity-relationship diagram



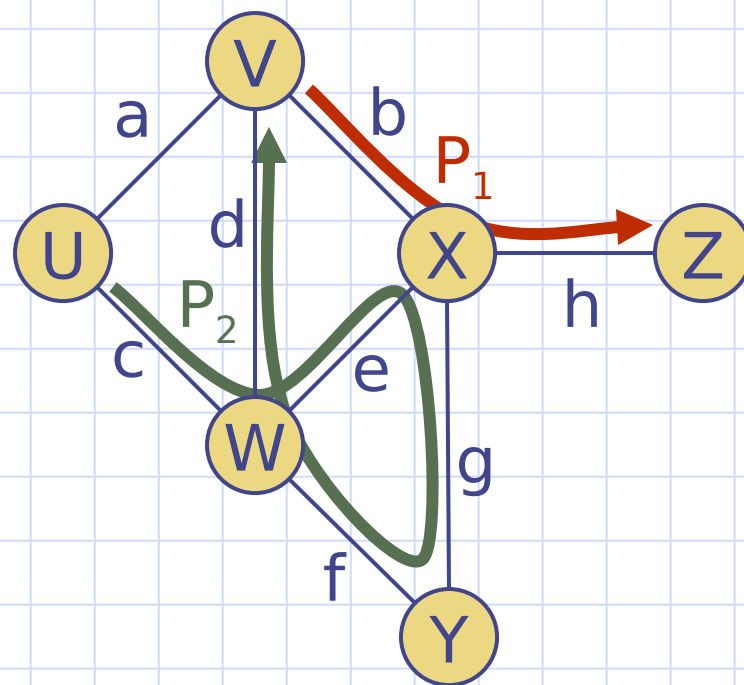
Terminology

- End vertices (or endpoints) of an edge
 - U and V are the endpoints of a
- Edges incident on a vertex
 - a, d, and b are incident on V
- Adjacent vertices
 - U and V are adjacent
- Degree of a vertex
 - X has degree 5
- Parallel edges
 - h and i are parallel edges
- Self-loop
 - j is a self-loop



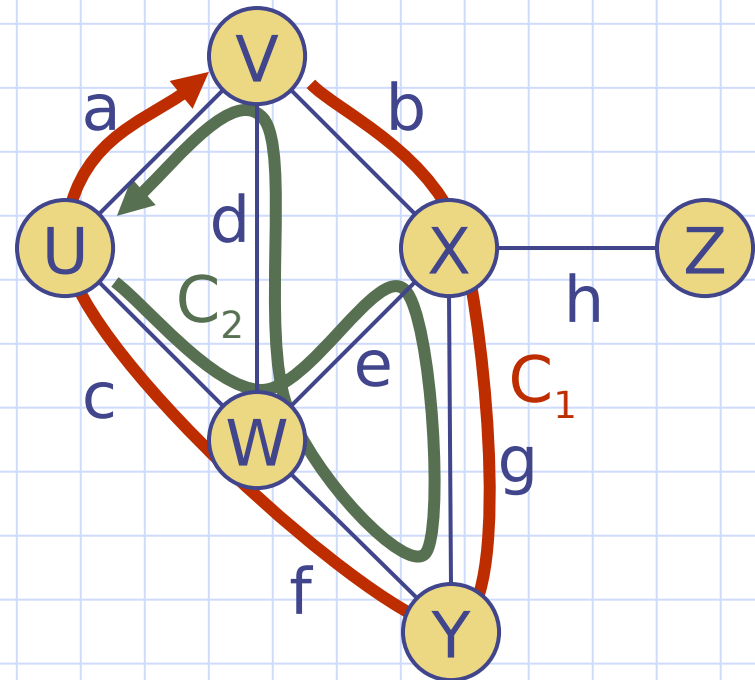
Terminology (cont.)

- Path
 - sequence of alternating vertices and edges
 - begins with a vertex
 - ends with a vertex
 - each edge is preceded and followed by its endpoints
- Simple path
 - path such that all its vertices and edges are distinct
- Examples
 - $P_1 = (V, b, X, h, Z)$ is a simple path
 - $P_2 = (U, c, W, e, X, g, Y, f, W, d, V)$ is a path that is not simple



Terminology (cont.)

- Cycle
 - circular sequence of alternating vertices and edges
 - each edge is preceded and followed by its endpoints
- Simple cycle
 - cycle such that all its vertices and edges are distinct
- Examples
 - $C_1 = (V, b, X, g, Y, f, W, c, U, a, \leftarrow)$ is a simple cycle
 - $C_2 = (U, c, W, e, X, g, Y, f, W, d, V, a, \leftarrow)$ is a cycle that is not simple



Properties

Property 1

$$\sum_v \deg(v) = 2m$$

Proof: each edge is counted twice

Property 2

In an undirected graph with no self-loops and no multiple edges

$$m \leq n(n-1)/2$$

Proof: each vertex has degree at most $(n-1)$

Notation

n number of vertices

m number of edges

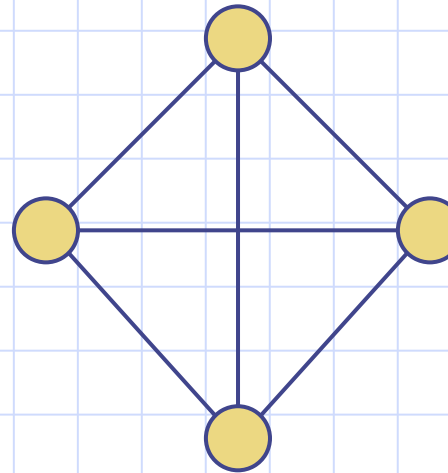
$\deg(v)$ degree of vertex

Example

■ $n = 4$

■ $m = 6$

■ $\deg(v) = 3$



What is the bound for a directed graph?

Vertices and Edges

- A **graph** is a collection of **vertices** and **edges**.
- We model the abstraction as a combination of three data types: Vertex, Edge, and Graph.
- A **Vertex** is a lightweight object that stores an arbitrary element provided by the user (e.g., an airport code)
 - We assume it supports a method, `element()`, to retrieve the stored element.
- An **Edge** stores an associated object (e.g., a flight number, travel distance, cost), retrieved with the `element()` method.
- In addition, we assume that an Edge supports the following methods:
 - `endpoints()`: Return a tuple (u, v) such that vertex u is the origin of the edge and vertex v is the destination; for an undirected graph, the orientation is arbitrary.
 - `opposite(v)`: Assuming vertex v is one endpoint of the edge (either origin or destination), return the other endpoint.

Graph ADT

`vertex_count()`: Return the number of vertices of the graph.

`vertices()`: Return an iteration of all the vertices of the graph.

`edge_count()`: Return the number of edges of the graph.

`edges()`: Return an iteration of all the edges of the graph.

`get_edge(u,v)`: Return the edge from vertex u to vertex v , if one exists; otherwise return `None`. For an undirected graph, there is no difference between `get_edge(u,v)` and `get_edge(v,u)`.

`degree(v, out=True)`: For an undirected graph, return the number of edges incident to vertex v . For a directed graph, return the number of outgoing (resp. incoming) edges incident to vertex v , as designated by the optional parameter.

`incident_edges(v, out=True)`: Return an iteration of all edges incident to vertex v . In the case of a directed graph, report outgoing edges by default; report incoming edges if the optional parameter is set to `False`.

`insert_vertex(x=None)`: Create and return a new Vertex storing element x .

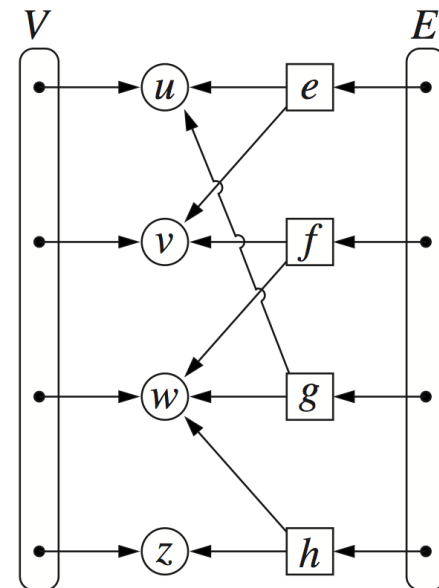
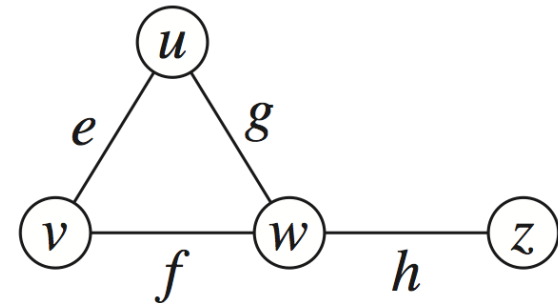
`insert_edge(u, v, x=None)`: Create and return a new Edge from vertex u to vertex v , storing element x (`None` by default).

`remove_vertex(v)`: Remove vertex v and all its incident edges from the graph.

`remove_edge(e)`: Remove edge e from the graph.

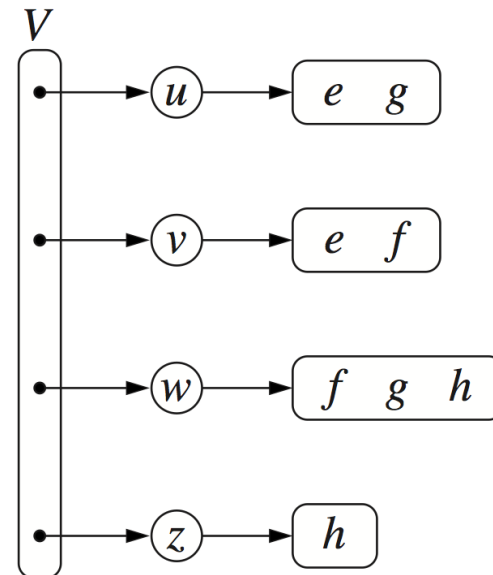
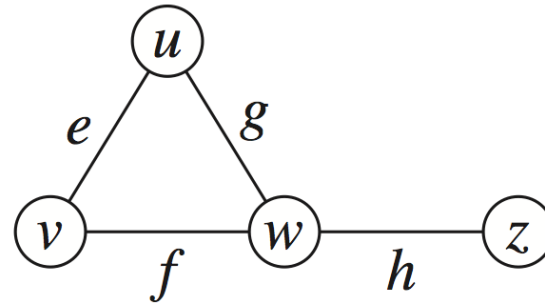
Edge List Structure

- Vertex object
 - element
 - reference to position in vertex sequence
- Edge object
 - element
 - origin vertex object
 - destination vertex object
 - reference to position in edge sequence
- Vertex sequence
 - sequence of vertex objects
- Edge sequence
 - sequence of edge objects



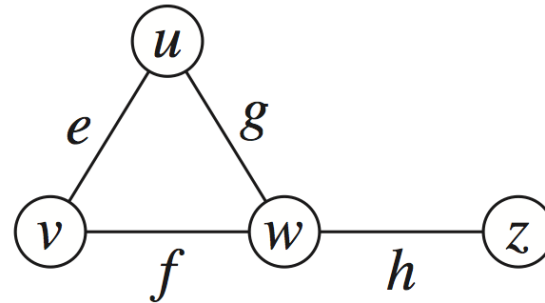
Adjacency List Structure

- Incidence sequence for each vertex
 - sequence of references to edge objects of incident edges
- Augmented edge objects
 - references to associated positions in incidence sequences of end vertices



Adjacency Matrix Structure

- Edge list structure
- Augmented vertex objects
 - Integer key (index) associated with vertex
- 2D-array adjacency array
 - Reference to edge object for adjacent vertices
 - Null for non adjacent vertices
- The “old fashioned” version just has 0 for no edge and 1 for edge



		0	1	2	3	
u	$\longrightarrow$	0		e	g	
v	$\longrightarrow$	1	e		f	
w	$\longrightarrow$	2	g	f		h
z	$\longrightarrow$	3			h	

Performance

▪ n vertices, m edges ▪ no parallel edges ▪ no self-loops	Edge List	Adjacency List	Adjacency Matrix
Space	$n + m$	$n + m$	n^2
incidentEdges (v)	m	$\text{deg}(v)$	n
areAdjacent (v , w)	m	$\min(\text{deg}(v), \text{deg}(w))$	1
insertVertex (o)	1	1	n^2
insertEdge (v , w , o)	1	1	1
removeVertex (v)	m	$\text{deg}(v)$	n^2
removeEdge (e)	1	1	1

Python Graph Implementation

- We use a variant of the ***adjacency map*** representation.
- For each vertex v , we use a Python dictionary to represent the secondary incidence map $I(v)$.
- The list V is replaced by a top-level dictionary D that maps each vertex v to its incidence map $I(v)$.
 - Note that we can iterate through all vertices by generating the set of keys for dictionary D .
- A vertex does not need to explicitly maintain a reference to its position in D , because it can be determined in $O(1)$ expected time.
- Running time bounds for the adjacency-list graph

Vertex Class

```
1  #----- nested Vertex class -----
2  class Vertex:
3      """Lightweight vertex structure for a graph."""
4      __slots__ = '_element'
5
6      def __init__(self, x):
7          """Do not call constructor directly. Use Graph's insert_vertex(x)."""
8          self._element = x
9
10     def element(self):
11         """Return element associated with this vertex."""
12         return self._element
13
14     def __hash__(self):          # will allow vertex to be a map/set key
15         return hash(id(self))
```


Edge Class

```
17 #----- nested Edge class -----
18 class Edge:
19     """ Lightweight edge structure for a graph. """
20     __slots__ = '_origin', '_destination', '_element'
21
22     def __init__(self, u, v, x):
23         """ Do not call constructor directly. Use Graph's insert_edge(u,v,x). """
24         self._origin = u
25         self._destination = v
26         self._element = x
27
28     def endpoints(self):
29         """ Return (u,v) tuple for vertices u and v. """
30         return (self._origin, self._destination)
31
32     def opposite(self, v):
33         """ Return the vertex that is opposite v on this edge. """
34         return self._destination if v is self._origin else self._origin
35
36     def element(self):
37         """ Return element associated with this edge. """
38         return self._element
39
40     def __hash__(self): # will allow edge to be a map/set key
41         return hash( (self._origin, self._destination) )
```